

Cryptographic certificates of validity for trustworthy AI

Murdoch J. Gabbay

Heriot-Watt University, Edinburgh, UK
m.gabbay@hw.ac.uk

Abstract

We propose cryptographic certificates of validity for agentic AI systems. The core idea is to formally specify a correctness or policy condition as a logical predicate, compile this predicate to a witness-checking problem over polynomial constraints, and use a succinct cryptographic proof system (and optionally zero-knowledge) to certify that the condition holds.

This offers a middle ground between formal verification of source code, and cryptographic authentication. An agent’s action can be accompanied by an independently checkable proof that it satisfies an agreed formal policy, without requiring the verifier to trust the agent or to re-execute computation.

We outline the approach at a high level, give the core mathematical translation, relate the proposal to proof-carrying code, zkVMs, formal methods, and agent governance, and note the specification, auditing, and deployment questions that a full implementation must answer.

1 Introduction

Agentic AI systems are beginning to take actions, not merely produce recommendations. A travel assistant may book a flight; an enterprise agent may approve an invoice; a software agent may deploy code; a robotic agent may act in the physical world. For such systems a key question is: *is this action authorised and policy compliant; is it correct?*

Cryptographic signatures traditionally authenticate *origin*, by certifying that the sender of a message knew a certain key. Logs can be used retrospectively to reconstruct events. However, cryptographic signatures, logs, and monitoring cannot prove that an action will satisfy correctness, safety, or compliance conditions *in advance*. In this paper, we propose a complementary mechanism: cryptographic certificates of *validity* for agent actions, before actions are carried out.

The idea is simple.

1. Express the relevant policy or correctness condition as a formal specification in an appropriate logic.
2. Compile the resulting finite witness-checking problem to cryptography-friendly polynomial constraints (as we will sketch).

3. Require the agent or an associated prover to attach a succinct cryptographic proof that the constraints are satisfied. (In zero-knowledge settings, the verifier may check the certificate without learning private witness data.)

The slogan is: *do not trust an action because of its origin; trust it because it carries cryptographically checkable evidence of correctness.*

A note on scope may now be helpful. The mathematics in Section 2 is application-agnostic: it provides an arithmetisation of first-order logic validity to judgements on polynomial constraints. This may be useful wherever a verifier requires succinct cryptographic evidence that a stated condition holds. This builds a bridge from first-order specification to cryptographic certification. The setting of agentic AI makes the assurance problem concrete and urgent — autonomous action is a signal case in which origin-based authentication and post-hoc logging may not be fully adequate. But the mathematical architecture which we use to address this problem — predicate $\rightarrow$ polynomial constraint $\rightarrow$ cryptographic certificate — is general, and might be usefully specialised to any domain where untrusted parties must provide checkable evidence of correctness.

From this perspective, systems of AI agents are just heterogeneous distributed systems of agents who may be independently developed and may act according to their own priorities. They may also (to borrow a term from distributed systems) be *Byzantine*, meaning that they do not satisfy the expected specification. Our proposal is to make selected actions proof-carrying, in a very particular sense: for their action to be accepted, they must produce a cryptographic certificate of validity of their correctness predicate.

Advanced AI assistants can plan and execute sequences of actions [Gabriel *et al.*, 2024]. Thus we need accountability, which requires evidence of where, why, how, and by whom agents are used [Chan *et al.*, 2024]. This paper offers a complementary assurance: further to identity, logging, and risk-management practices such as those articulated in the NIST AI Risk Management Framework [National Institute of Standards and Technology, 2023], selected agent actions should carry compact certificates that a formal specification has been validated. We can think of this as a cryptographic analogue of proof-carrying code [Necula, 1997], instantiated using succinct arguments and polynomial arithmetisation [Thaler, 2022; Gabbay, 2024; Gabbay, 2025;

$$t ::= X \mid a \in \mathbb{Z} \mid t + t \mid t * t \mid C_i(t) \mid \text{len}(C) \mid \text{reify}(\phi) \\ \phi, \psi ::= t = t \mid \phi \wedge \phi \mid \phi \vee \phi \mid \forall_C X. \phi \mid C_i < t \mid t < C_i$$

Above, $i \in [\text{arity}(C)]$. These grammars are in BNF style, such that ϕ and t to the right of $::=$ represent any predicate or term. Thus $\phi \wedge \phi$ denotes a conjunction of two (possibly non-equal) predicates, and $t = t$ denotes two (possibly non-equal) terms.

Figure 1: Syntax of terms and predicates (Definition 2.3)

$$\begin{aligned} [X]_\varsigma &= X & [a]_\varsigma &= a \\ [t + t']_\varsigma &= [t]_\varsigma + [t']_\varsigma & [t * t']_\varsigma &= [t]_\varsigma * [t']_\varsigma \\ [C_i(t)]_\varsigma &= \text{itplnt}(\varsigma(C)_i)([t]_\varsigma) & [\text{len}(C)]_\varsigma &= \text{len}(\varsigma(C)) \\ [\text{reify}(\phi)]_\varsigma &= [\phi]_\varsigma \\ [t = t']_\varsigma &= ([t]_\varsigma - [t']_\varsigma)^2 \\ [\phi \wedge \phi']_\varsigma &= [\phi]_\varsigma + [\phi']_\varsigma & [\phi \vee \phi']_\varsigma &= [\phi]_\varsigma * [\phi']_\varsigma \\ [\forall_C X. \phi]_\varsigma &= \sum_{x \in [\text{len}(\varsigma(C))]} [\phi]_\varsigma(x) \\ [C_i < t]_\varsigma &= \begin{cases} 0 & \forall j \in [\text{len}(\varsigma(C))]. (\varsigma(C)_{i,j} < [t]_\varsigma(j)) \\ 1 & \text{otherwise} \end{cases} \\ [t < C_i]_\varsigma &= \begin{cases} 0 & \forall j \in [\text{len}(\varsigma(C))]. ([t]_\varsigma(j) < \varsigma(C)_{i,j}) \\ 1 & \text{otherwise} \end{cases} \end{aligned}$$

In clauses mentioned in C_i above, we insist on $i \in [\text{arity}(C)]$ for well-formedness.

Figure 2: Polynomial semantics for terms and predicates (Definition 2.7)

Garreta et al., 2025].

One final note, for the record: a certificate of validity for a formal specification does not in and of itself prove that the formal specification is the *right* specification. It proves, under the stated assumptions and to within appropriate soundness bounds of the underlying cryptographic back end, that the formal specification has been validated.

2 A compact slice of maths

Notation 2.1. Define $[n] = \{1, \dots, n\} \subseteq \mathbb{N}$. If R is a ring and X is a formal **variable symbol** then $P \in R[X]$ is a formal sum of powers of X , which we call a **polynomial** (with coefficients in R). Polynomials can be added and multiplied in the usual way. For example, $(1+X)*(1-X) = 1-X^2 \in \mathbb{Z}[X]$.

Notation 2.2. If $P, Q \in \mathbb{Z}[X]$ then write $P(Q) \in \mathbb{Z}[X]$ for the result of **instantiating** X to Q in P . For example if $P = X + 1$ and $Q = 2 * X$ then $P(Q) = 2 * X + 1$.

Definition 2.3. Fix an **index variable symbol** X and a set of **polynomial function symbols** $C, D \in \text{PolyFunc}$, each of which has a fixed but arbitrary **arity** $\text{arity}(C) \in \mathbb{N}_{\geq 1}$. Write $C : n$ for the assertion “ $C \in \text{PolyFunc}$ and $\text{arity}(C) = n$ ”.

Define **terms** and **predicates** inductively as in Figure 1.

Notation 2.4. We may write $t \leq C_i \leq t'$ for $(t - 1 < C_i) \wedge (C_i < t' + 1)$. We may call predicates based on $<$ **range-checks**.

Notation 2.5. An **integer matrix** is an element $C \in \mathbb{Z}^{ar \times ln}$ where $ar \in \mathbb{N}$ (for **arity**) is the number of **rows** and $ln \in \mathbb{N}$ (for **length**) is the number of **columns**. Write $\text{len}(C)$ for ln the number of columns in C , which we may call the **length** of C . Write C_i for the i th row in C (a vector of length $\text{len}(C)$). Write $C_{i,j}$ for the i, j -th element in C ; i is the row number and j is the column number.

Definition 2.6. Suppose $\vec{a} = (a_1, \dots, a_n) \in \mathbb{Z}^{ln}$ is a ln -tuple of integers, for $ln \in \mathbb{N}$. Say that $P \in \mathbb{Q}[X]$ **interpolates** $\vec{a}$ at $(1, 2, \dots, ln)$ when $\forall x \in [ln]. P(x) = a_x$.

It is a fact [Salgado and Wise, 2022, Chapter 9] that every $\vec{a} \in \mathbb{Z}^{ln}$ can be interpolated by a unique **polynomial interpolant** $\text{itplnt}(a_1, \dots, a_{ln}) \in \mathbb{Q}[X]$ of degree at most $ln-1$, so $\text{itplnt}(a_1, \dots, a_{ln})(x) = a_x$.¹

Definition 2.7.

1. An **interpretation** ς assigns to $C \in \text{PolyFunc}$ a matrix $\varsigma(C) \in \mathbb{Z}^{\text{arity}(C) \times \text{len}(\varsigma(C))}$. $\varsigma(C)$ always has $\text{arity}(C)$ rows; the number of columns depends on ς .
2. Define **semantics** $[t]_\varsigma, [\phi]_\varsigma \in \mathbb{Q}[X]$ as in Figure 2.
3. If $x \in \mathbb{Z}$ then define the **(validity) judgement** $x \models_\varsigma \phi$ to mean $[\phi]_\varsigma(x) = 0$.
If ϕ is closed (has no free variables) then define the **(validity) judgement** $\models_\varsigma \phi$ to mean $[\phi]_\varsigma(0) = 0$.²

Lemma 2.8. $[\phi]_\varsigma(x) \geq 0$ for every $x \in \mathbb{Q}$, and in particular for every $x \in \mathbb{Z}$.

Proof. Routine induction on Figure 2, noting that: for $x, y \in \mathbb{Q}$, $(x - y)^2 \geq 0$; and for $x, y \geq 0$, $x + y, x * y \geq 0$. $\square$

We can now prove *soundness* and *completeness*; validity with respect to the predicate connectives in Figure 1 behaves exactly as those symbols suggest:

Theorem 2.9 ([Gabbay, 2024; Gabbay, 2025]).

1. $x \models_\varsigma t = t'$ if and only if $[t]_\varsigma(x) = [t']_\varsigma(x)$.
2. $x \models_\varsigma \phi \wedge \phi'$ if and only if $x \models_\varsigma \phi \wedge x \models_\varsigma \phi'$ and $x \models_\varsigma \phi \vee \phi'$ if and only if $x \models_\varsigma \phi \vee x \models_\varsigma \phi'$.
3. $x \models_\varsigma \forall_C X. \phi$ if and only if $x' \models_\varsigma \phi$ for every $x' \in [\text{len}(\varsigma(C))]$.
4. $x \models_\varsigma t < C_i$ if and only if $\varsigma(C)_{i,j}$ is strictly greater than $[t]_\varsigma(j)$ for every $j \in [\text{len}(\varsigma(C))]$, and similarly for $x \models_\varsigma C_i < t$.

Proof. Routine from Figure 2 and Lemma 2.8, noting that: for $x, y \in \mathbb{Q}$, $(x-y)^2 = 0$ iff $x = y$; and for $x, y \geq 0$, $x + y = 0$ iff $x=0 \wedge y=0$ and $x * y = 0$ iff $x=0 \vee y=0$. $\square$

3 Examples

3.1 Three one-line examples

Section 2 takes up one page of dense mathematics and concludes with one Theorem. This is compact and theoretical

¹The [Wikipedia page](#) and [this overview](#) are excellent and very accessible introductions.

²The choice of evaluation point 0 is unimportant; we could equivalently use $[\phi]_\varsigma(1)$, since for closed ϕ , $\forall x, x'. [\phi]_\varsigma(x) = [\phi]_\varsigma(x')$.

Logic	Polynomial
Truth values	0 = true/valid/success; strictly positive = false/invalid/failure.
Equality	$[t = t']_\varsigma = ([t]_\varsigma - [t']_\varsigma)^2$, so equality is valid exactly at zero.
Conjunction	$[\phi \wedge \psi]_\varsigma = [\phi]_\varsigma + [\psi]_\varsigma$; sum of non-negatives is zero iff both are zero.
Disjunction	$[\phi \vee \psi]_\varsigma = [\phi]_\varsigma * [\psi]_\varsigma$; product is zero iff at least one factor is zero.
Range check	0 when satisfied, 1 otherwise.
Top-level $\forall$	Root test for a range of values.
ϕ valid	$[\phi]_\varsigma = 0$
ϕ invalid	$[\phi]_\varsigma > 0$

Figure 3: Table of semantic meaning (Remark 3.2)

much as logic gates are compact and theoretical: by composing together simple components we attain great expressivity, and by suitable implementation we can accomplish remarkable things. We give the reader a taste of how this works.

Example 3.1. We can express truth as $0=0$ and false as $0=1$. The reader can check that $[0=0]_\varsigma = 0$ (thus $\models_\varsigma 0=0$) and $[0=1]_\varsigma = 1$ (thus $\not\models_\varsigma 0=1$) so these predicates do behave like ‘true’ and ‘false’.

Assume $m : 2$, $a : 3$, and $p : 1$ with *validity predicates*

$$\begin{aligned}\chi_m &= \forall_m X. (m_2(X) = (-1) * m_1(X)) \\ \chi_a &= \forall_a X. (a_3(X) = a_1(X) + a_2(X)) \\ \chi_p &= (0 < p_1).\end{aligned}$$

Given an interpretation ς , we have

$$\begin{aligned}[\chi_m]_\varsigma &= \sum_{x \in [len(\varsigma(m))]} (itplnt(m_2)(x) - (-1) * itplnt(m_1)(x))^2 \\ [\chi_a]_\varsigma &= \sum_{x \in [len(\varsigma(a))]} (itplnt(a_3)(x) - (itplnt(a_1)(x) + itplnt(a_2)(x)))^2 \\ [\chi_p]_\varsigma &= 0 \text{ if } \forall x \in [len(\varsigma(p))] . \varsigma(p)_{1,x} > 0, \text{ and } 1 \text{ otherwise}\end{aligned}$$

So:

- $\models_\varsigma \chi_m$ when $\varsigma(m)$ is a two-row matrix where in each column the second entry is the negation of the first. Thus, each column in $\varsigma(m)$ represents a pair $(x, -x)$ for some $x \in \mathbb{Z}$.
arity(m) = 2 because $\lambda x. -x$ has one input and output.
- $\models_\varsigma \chi_a$ when $\varsigma(a)$ is a three-row matrix where in each column the third entry is the sum of the first and the second; thus $\varsigma(a)$ has columns of the form $(x, x', x+x')$ and samples the graph of $\lambda x, x'. x+x'$ at $len(\varsigma(a))$ many points.
arity(a) = 3 because $\lambda x, x'. x+x'$ has two inputs and one output.
- $\models_\varsigma \chi_p$ when $\varsigma(p)$ is a one-row matrix (a vector) of strictly positive numbers.
arity(p) = 1 because ‘is strictly positive’ is a predicate on one argument.

Remark 3.2. A table of semantic meanings is in Figure 3.

The reader may be more used to seeing ‘true = 1; false = 0’, but representing truth with 0 and non-truth with strictly positive values also has pedigree. In a paper from 1943 [Kleene, 1943] on page 51 just after equation 17, Kleene writes “a

representing function π of P , the value of which is to be 0, 1, or undefined according as the value of P is true, false, or undefined”. More recently, in the `sysexits.h` file (credited to Eric Allman in 1980) which describes status codes for system programs, 0 represents successful termination and strictly positive values represent various kinds of failure.

We will consider one more example, because it is an important one. We will indicate that our framework can handle *recursion*; this is the essence of looping and of Turing-complete computation. The reader will have to excuse that our example will be extremely simple, but it is the *structure* of the computation that interests us.

3.2 A recursive example

Definition 3.3. Recall the inductive definition of a^b for $a, b \in \mathbb{N}_{\geq 0}$:

$$\begin{array}{ll}\text{base case (BC)} & pow(a, 0) = 1 \\ \text{inductive step (IS)} & pow(a, b+1) = a * pow(a, b)\end{array}$$

Remark 3.4. Note how Definition 3.3, and definitions like it, implicitly equate *computation*, *inductively-defined relations*, and *derivation*, because *pow* is a relation, it is inductively defined as above, and when we try to compute a value — *pow*(2, 2), say — we generate a derivation-tree as follows:

$$\begin{array}{l} \text{base case} \\ \hline pow(2, 0) = 1 \\ \text{inductive step} \\ \hline pow(2, 1) = 2 \\ \text{inductive step} \\ \hline pow(2, 2) = 4 \end{array}$$

This example, simple as it is, illustrates the close link between logic, computation, and inductively defined relations.

Definition 3.5. Suppose $ln \in \mathbb{N}_{\geq 1}$. Say a $4 \times ln$ matrix C encodes a partial power function when

$$\forall a \in 1..ln. C_{3,a} = C_{1,a}^{C_{2,a}}.$$

In words, C encodes a partial power function when for each column, the third entry is equal to the first entry raised to the power of the second.

Definition 3.6. Assume $pow : 4$, so $\varsigma(pow) \in \mathbb{Z}^{4 \times len(\varsigma(pow))}$. Figure 4 expresses a validity predicate χ_{pow} , that *pow* encodes a partial power function such that:

- Row 1 (values of $pow_1(a)$) stores input values a .
- Row 2 (values of $pow_2(a)$) stores input values b .
- Row 3 (values of $pow_3(a)$) stores output values a^b .
- Row 4 (values of $pow_4(a)$) stores the index of a column representing the recursive call in the inductive step.

Lemma 3.7. If $\models_\varsigma \chi_{pow}$ then $\varsigma(pow)$ encodes a partial power function.

Proof. We examine the clauses in Figure 4 and using Theorem 2.9 we check that they correctly encode the definition in Definition 3.3. $\square$

Write $\text{pow}_1(t)=\text{in}_1(t)$, $\text{pow}_2(t)=\text{in}_2(t)$, $\text{pow}_3(t)=\text{out}(t)$, and $\text{pow}_4(t)=\text{rec}(t)$.

$$\begin{aligned} \text{BC}(X) &= (\text{in}_2(X) = 0) \wedge (\text{out}(X) = 1) \\ \text{IS}(X) &= \text{in}_1(X) = \text{in}_1(\text{rec}(X)) \wedge \text{in}_2(X) = \text{in}_2(\text{rec}(X)) + 1 \wedge \\ &\quad \text{out}(X) = \text{in}_1(X) * \text{out}(\text{rec}(X)) \\ \chi_{\text{pow}} &= 1 \leq \text{rec} \leq \text{len}(\text{pow}) \wedge \forall_{\text{pow}.X}. (\text{BC}(X) \vee \text{IS}(X)) \end{aligned}$$

Figure 4: Exponentiation a^b (Definitions 3.3 and 3.6)

Example 3.8. We illustrate some values for $\varsigma(\text{pow})$ such that $\models_{\varsigma} \chi_{\text{pow}}$, so that by Lemma 3.7 they encode a partial power function:

$$\begin{pmatrix} 2 & 2 & 2 \\ 0 & 1 & 2 \\ 1 & 2 & 4 \\ 1 & 1 & 2 \end{pmatrix} \quad \begin{pmatrix} 2 & 2 & 2 \\ 2 & 1 & 0 \\ 4 & 2 & 1 \\ 2 & 3 & 3 \end{pmatrix} \quad \begin{pmatrix} 2 & 2 & 2 & 2 \\ 0 & 2 & 1 & 0 \\ 1 & 4 & 2 & 1 \\ 2 & 3 & 1 & 1 \end{pmatrix} \quad \begin{pmatrix} 3 & 2 & 3 & 2 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 3 & 2 \\ 1 & 1 & 1 & 2 \end{pmatrix}$$

These all represent the derivation-tree from Remark 3.4:

1. The leftmost matrix encodes in columns 1-3 respectively, the base case of a derivation that $2^0 = 1$, the step $2^1 = 2$, and the final step $2^2 = 4$. Entry 4 in columns 2 and 3 contain a number which is a pointer to the previous step/column in the derivation; entry 4 in column 1 also contains a number but it is not used.
2. The columns need not appear in a particular order, provided that the pointers match up; the next matrix illustrates this by putting the columns in the reverse order.
3. Columns can be duplicated as per the third matrix.
4. The fourth (rightmost) matrix includes information from computing $2^1 = 2$ and $3^1 = 3$.

Example 3.9. The converse implication for Lemma 3.7 does not hold. The predicate χ_{pow} certifies the pointwise correctness of the input-output samples in rows 1-3 *and also* that those samples can be assembled into a well-formed derivation tree, whose recursive calls are represented by in-range pointers in row 4.

A matrix may encode true values of the partial power function while failing χ_{pow} , if the certificate of derivation is missing or malformed. For example, the one-column matrix to the right contains the correct input-output sample (2,2,4), since $2^2 = 4$. However, it omits the recursive premise (2,1,2) required by the inductive step. Formally, $\text{len}(\text{pow}) = 1$ and the row 4 entry is 2, so the range check $1 \leq \text{rec} \leq \text{len}(\text{pow})$ fails. Equivalently: the supposed recursive pointer points outside the matrix.

A second instructive failure is as per the three-column matrix to the right. Rows 1-3 contain correct samples: $2^2 = 4$, $2^1 = 2$, and $2^0 = 1$, and the first two recursive pointers are meaningful: column 1 points to column 2, and column 2 points to column 3. However, the underlined pointer in the base-case column is not in the range $\{1, 2, 3\}$. The base-case clause (BC) does not use this pointer, but our simple presentation imposes the uniform range check on all entries of the pointer row. Hence this matrix encodes correct samples of the partial power function but is not a well-formed χ_{pow} -certificate. Replacing the underlined 0 with any in-range value, for example with 1, would repair this.

In summary: rows 1-3 describe claimed values of the relation; row 4 supplies the proof-carrying structure that allows a verifier to check claims against the inductive definition.

Remark 3.10. Example 3.9 connects directly to agent certification. Columns are not isolated output values; they are structured witnesses which reference each other to prove their validity. In an agentic setting this distinction separates a validator that checks whether an action is compliant from one that checks whether an action carries evidence of its compliance (in this case, columns pointing to one another to prove a recursive computation). This is precisely the proof-carrying structure that the certificates of Section 4 would cryptographically attest to.

Remark 3.11. A more efficient specification of a^b is:

$$\begin{aligned} \text{pow}(a, 0) &= 1 \\ \text{pow}(a, 2 * b) &= \text{pow}(a, b) * \text{pow}(a, b) \\ \text{pow}(a, 2 * b + 1) &= a * \text{pow}(a, b) * \text{pow}(a, b) \end{aligned}$$

This yields smaller derivations, which correspond algorithmically to a runtime that is logarithmic in b rather than linear. We could translate this into our logic, and it would just yield an implementation that permits shorter matrices.

Remark 3.12. The logic in Figure 1 is more expressive than it might seem. In particular, negation is expressible in the framework, via reify. A fuller treatment is in [Gabbay, 2025]. A proposal for a SNARK-friendly constraint system is in [Gabbay and Mendelsohn, 2026].

4 How to apply this mathematics to AI agents

Before we address the question of how to apply this mathematics to AI agents — the motivating application of this paper, although the framework in Sections 2 and 3 is general — it is important to understand that polynomial semantics can be plugged directly into cryptographic proof schemes.

The semantics in Figure 2 is optimised for exposition, but variations optimised for implementation can be constructed.³

Thus it is important to appreciate that the presentation in Section 2 is optimised for fitting onto one page including Figure 2 and Theorem 2.9, consistent with clear exposition. We can optimise for practical implementability instead, but that would be a longer and less accessible presentation: the passage from cleanly presentable theory to applicability is analogous to the passage from a high-level programming language to something more like assembly language or bytecode. Be that as it may, the strategy for applying these ideas is simple:

- Write down a predicate that expresses the desired notion of correctness, and translate it (using techniques analogous to program compilation) to an algebraic relation accepted by a cryptographic proof back end. Suitable back ends include integer or mixed-characteristic systems such as Zinc [Garreta et al., 2025] and polynomial-commitment/SNARK stacks such as Halo2 and related foundations [Electric Coin Company and Zcash Foundation, 2026; Boneh et al., 2021].

³One such variant has been submitted for publication, but at time of writing, submission rules prohibit sharing the results. By time of publication I hope to be able to provide links.

- Require the agent, or associated prover, to produce a cryptographic certificate accepted by the back end. Under the proof system’s assumptions, and subject to stated soundness error, a valid certificate convinces the verifier that the encoded relation has a witness.

If this seems conceptually simple, that is because it is.

Though this is a short paper, we can make this more concrete. At the boundary of an agentic system, we can represent a certificate of validity abstractly as

$$\text{Cert} = (\text{policyID}, \text{action}, \text{pub}, \text{vk}, \text{paramsHash}, \pi).$$

Here policyID identifies the formal predicate ϕ and the compiler version; action is the proposed action; pub is the public instance data needed to check that action; vk and paramsHash identify the approved verifier key and proof-system parameters; and π is the succinct cryptographic proof. The intended statement verified by π is

$$\exists w. R_{\phi, \text{action}}(\text{pub}, w) = 0,$$

where w is private witness data and $R_{\phi, \text{action}}$ is the algebraic relation obtained by compiling the formal predicate to polynomial constraints.

Thus verification of an action has the following form.

1. The policy author or system operator fixes a predicate ϕ expressing the required correctness, safety, or compliance condition.
2. A compiler translates ϕ , together with the relevant public action data, into an algebraic relation $R_{\phi, \text{action}}$.
3. The agent, or an associated prover, produces a proof π that there exists witness data w satisfying that relation.
4. The receiving system checks π using the approved verifier key and rejects action if verification fails.

We can sum this up as a slogan: *logical validity translates to vanishing of the corresponding algebraic relation.*

In this interface the verifier need not trust the agent, inspect internal implementation, or re-execute computation. It just checks the proof. Conversely, the certificate proves only the compiled formal claim: it does not prove that the policy was wisely chosen, that the compiler is bug-free, or that the proof-system parameters were correctly governed.

A practical compiler would use implementation-oriented arithmetisation techniques; for example multilinear encodings, range checks, and lookup arguments, targeting the back ends mentioned above (Zinc; Halo2). The mathematics of Sections 2 and 3 specifies the correctness condition that such a compiler must preserve.

Roadmap. So we can propose a *provisional roadmap*. If a policy can be logically expressed as a predicate, and if a trusted compilation chain maps that predicate to an algebraic relation while preserving the zero-versus-positive philosophy of Figure 3, then any sound succinct argument for that relation yields a succinct certificate that the policy predicate is satisfied by the supplied witness data. Remaining questions are just what one would expect, moving from mathematics to a deployed assurance mechanism: how to author and audit correctness predicates; how to verify or test the compiler;

how to select and parameterise the proof back end; how to benchmark proving cost; and how to record policy versions and verifier configurations for later accountability.

5 Related work

Formal specification and verification. Formal methods provide mature tools for specifying and proving properties of software and systems. Hoare logic [Hoare, 1969], proof-carrying code [Necula, 1997], verified compilers like CompCert [Leroy, 2009], and verified kernels like seL4 [Klein et al., 2009] can provide machine-checkable assurance for critical systems. *Correctness-by-construction* makes assurance part of design and implementation [Hall and Chapman, 2002]. These approaches share with our proposal a formally specified notion of ‘correctness’.

However, the deployments differ: proof-carrying code is not a black-box technique, whereas our proposal imposes no restrictions on how an agent is designed and implemented, other than insisting that some or all of the actions that it proposes be associated to a succinct certificate proving validity of an agreed correctness predicate.

Runtime verification, proof-carrying code, and certificates. *Runtime verification* checks traces of an executing system against formal properties [Leucker and Schallhart, 2009]. *Proof-carrying code* asks an untrusted code producer to supply a proof that code satisfies a consumer’s safety policy [Necula, 1997].

Our proposal can be read as a cryptographic, succinct, and optionally zero-knowledge analogue of proof-carrying code: the consumer publishes or fixes a policy; the agent supplies an action and a certificate; and the verifier checks the certificate without re-running the agent’s entire computation or inspecting all private witness data.

Succinct cryptographic proofs and verifiable computation. *zk-SNARKs* and related succinct arguments allow a *prover* to convince a *verifier* that a relation holds between a *witness* and an *instance*, while keeping verifier effort small and, in zero-knowledge variants, hiding some or all of the witness. Examples include Pinocchio [Parno et al., 2013], SNARKs for C [Ben-Sasson et al., 2013], Groth’s pairing-based arguments [Groth, 2016], and later general frameworks for polynomial IOPs, constraint systems, and related lookup machinery [Ben-Sasson et al., 2016; Bünz et al., 2020; Thaler, 2022; Setty et al., 2023; Setty et al., 2024]. The technical constructions underlying this paper use similar technology; in unpublished work we have shown how to arithmetise the first-order logic of this paper to polynomial constraints on integer/mixed-characteristic proof systems such as Zinc [Garreta et al., 2025], and we are studying how to translate to Halo2 [Electric Coin Company and Zcash Foundation, 2026] so that logical specifications can be connected to cryptographic back ends.

zkVMs and proof-oriented execution environments. An engineering response to the difficulty of writing polynomial constraints directly is zero-knowledge virtual machines, like Cairo [Goldberg et al., 2021], RISC0 [RISC Zero, 2026], and Jolt [Arun et al., 2024]. These all offer verifiable receipts of

correct code execution. Work on formally verifying Cairo execution in Lean reflects the importance of connecting cryptographic proof systems to proof-assistant-level assurance [Avigad *et al.*, 2021].

In contrast, the present proposal does not introduce another machine model. Its high-level specification language is first-order logic itself. Programs, policies, transaction rules, and compliance requirements are represented as predicates, which are compiled directly to polynomial proof obligations.

Choreographic programming and distributed correctness. Choreographic programming and multiparty session types address distributed correctness from complementary angles. Multiparty session types use global protocol descriptions whose projections type-check communicating processes and establish communication safety, progress, and fidelity [Honda *et al.*, 2016]. Choreographic programming writes a global coordination plan from which decentralised implementations can be generated correct by construction [Cruz-Filipe and Montesi, 2017] (subsequently developed to include functional [Cruz-Filipe *et al.*, 2022] and machine-checked [Cruz-Filipe *et al.*, 2023] formulations). These ideas may be relevant to agentic AI systems. Our proposal does not replace choreographic or protocol-based design, and it deals with the orthogonal problem of how to ensure that possibly Byzantine, possibly black-box agents — i.e. agents which we did not implement and which we may not be able to look inside — should still carry cryptographic certificates of correct behaviour.

6 Conclusions

Formal methods provide the language of formal specification; correctness-by-construction provides a design discipline; runtime verification and proof-carrying code provide certificate-oriented assurance; and zk-SNARKs/zkVMs provide cryptographically checkable execution.

This paper articulates a synthesis of these ideas for trustworthy agentic AI: a consequential action may be logged and signed — *and also*, where feasible, it may be accompanied by a succinct certificate of validity of a formally specified notion of correctness, safety, or compliance.

It remains to render these ideas into cryptographically implementable schemes, e.g. by reduction to SNARK-friendly constraint systems. This is ongoing work: a proposal has been very recently presented [Gabbay and Mendelsohn, 2026] and a conference paper submitted.

Future work includes generating auditable deployment patterns; e.g. specification templates for common agent actions, benchmarks across proof back ends, recording of policy versions and verifier configurations, and independent cryptographic, formal-methods, and policy review.

These are not mere engineering details. A cryptographic certificate proves satisfaction of the encoded formal predicate under the relevant proof-system assumptions and soundness bounds. It does *not* prove that the correctness predicate itself reflects safety, legal, operational, or ethical requirements. This specification gap must be addressed through governance, visibility, and risk-management processes; it cannot be closed

by mathematics, logic or cryptography alone [Chan *et al.*, 2024; National Institute of Standards and Technology, 2023].

References

- [Arun *et al.*, 2024] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: SNARKs for virtual machines via lookups. In *Advances in Cryptology – EUROCRYPT 2024*, volume 14656 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2024. doi:10.1007/978-3-031-58751-1_1.
- [Avigad *et al.*, 2021] Jeremy Avigad, Lior Goldberg, David Levit, Yoav Seginer, and Alon Titelman. A verified algebraic representation of cairo program execution, 2021. doi:10.48550/arXiv.2109.14534.
- [Ben-Sasson *et al.*, 2013] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In *Advances in Cryptology – CRYPTO 2013*, volume 8043 of *Lecture Notes in Computer Science*, pages 90–108. Springer, 2013. doi:10.1007/978-3-642-40084-1_6.
- [Ben-Sasson *et al.*, 2016] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2016. doi:10.1007/978-3-662-53644-5_2.
- [Boneh *et al.*, 2021] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. Halo infinite: Proof-carrying data from additive polynomial commitments. In *Advances in Cryptology – CRYPTO 2021*, volume 12825 of *Lecture Notes in Computer Science*, pages 649–680. Springer, 2021. doi:10.1007/978-3-030-84242-0_23.
- [Bünz *et al.*, 2020] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In *Advances in Cryptology – EUROCRYPT 2020*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706. Springer, 2020. doi:10.1007/978-3-030-45721-1_24.
- [Chan *et al.*, 2024] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. Visibility into AI agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, FAccT ’24, page 958–973, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3630106.3658948.
- [Cruz-Filipe and Montesi, 2017] Luís Cruz-Filipe and Fabrizio Montesi. A core model for choreographic programming. In *Formal Aspects of Component Software*, volume 10231 of *Lecture Notes in Computer Science*, pages 17–35. Springer, 2017. doi:10.1007/978-3-319-57666-4_3.
- [Cruz-Filipe *et al.*, 2022] Luís Cruz-Filipe, Eva Graversen, Lovro Lugović, Fabrizio Montesi, and Marco Peressotti. Functional choreographic programming. In *Theoretical Aspects of Computing – ICTAC 2022*, Lecture Notes in Computer Science. Springer, 2022. doi:10.1007/978-3-031-17715-6_15.

- [Cruz-Filipe *et al.*, 2023] Luís Cruz-Filipe, Fabrizio Montesi, and Marco Peressotti. A formal theory of choreographic programming. *Journal of Automated Reasoning*, 67, 2023. doi:10.1007/s10817-023-09665-3.
- [Electric Coin Company and Zcash Foundation, 2026] Electric Coin Company and Zcash Foundation. Halo2: The Halo2 zero-knowledge proving system, 2026. Software repository; accessed 2 May 2026. <https://github.com/zcash/halo2>.
- [Gabbay and Mendelsohn, 2026] Murdoch J. Gabbay and Andrew Mendelsohn. SNARKs for First Order Logic. Talk presented by Mendelsohn at the [Workshop on Cryptographic Tools for Blockchains](https://www.ctb-workshop.org/slides/02-SNARKs-for-First-Order-Logic.pdf) (CTB 2026, [perma-link](https://www.ctb-workshop.org/slides/02-SNARKs-for-First-Order-Logic.pdf)), affiliated with Eurocrypt 2026, Rome, Italy, May 2026. Slides at <https://www.ctb-workshop.org/slides/02-SNARKs-for-First-Order-Logic.pdf> ([perma-link](https://www.ctb-workshop.org/slides/02-SNARKs-for-First-Order-Logic.pdf)).
- [Gabbay, 2024] Murdoch J. Gabbay. Arithmetisation of computation via polynomial semantics for first-order logic. Cryptology ePrint Archive, Paper 2024/954, 2024. <https://eprint.iacr.org/2024/954>.
- [Gabbay, 2025] Murdoch J. Gabbay. Arithmetising logic: polynomial semantics of FOL. *Journal of Applied Logics*, 12(6), October 2025. <http://www.collegepublications.co.uk/downloads/ifcolog00074.pdf#page=107>.
- [Gabriel *et al.*, 2024] Iason Gabriel, Arianna Manzini, Geoff Keeling, Lisa Anne Hendricks, Verena Rieser, Hasan Iqbal, Nenad Tomašev, Ira Ktena, Zachary Kenton, Mikel Rodriguez, et al. The ethics of advanced ai assistants, 2024. doi:10.48550/arXiv.2404.16244.
- [Garreta *et al.*, 2025] Albert Garreta, Hendrik Waldner, Ilia Vlasov, Katerina Hristova, Luca Dall’Ava, Marko Čupić, and Matthew Klein. Zinc: Succinct arguments with small arithmetization overheads from IOPs of proximity to the integers. In *Advances in Cryptology – CRYPTO 2025*, volume 16006 of *Lecture Notes in Computer Science*, pages 259–291. Springer Nature Switzerland, 2025. doi:10.1007/978-3-032-01907-3_9.
- [Goldberg *et al.*, 2021] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a Turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Paper 2021/1063, 2021. <https://eprint.iacr.org/2021/1063>.
- [Groth, 2016] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016. doi:10.1007/978-3-662-49896-5_11.
- [Hall and Chapman, 2002] Anthony Hall and Roderick Chapman. Correctness by construction: Developing a commercial secure system. *IEEE Software*, 19(1):18–25, 2002. doi:10.1109/52.976937.
- [Hoare, 1969] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi:10.1145/363235.363259.
- [Honda *et al.*, 2016] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. *Journal of the ACM*, 63(1):1–67, 2016. doi:10.1145/2827695.
- [Kleene, 1943] Stephen C. Kleene. Recursive predicates and quantifiers. *Transactions of the American Mathematical Society*, 53:41–73, 1943. doi:10.1090/S0002-9947-1943-0007371-8.
- [Klein *et al.*, 2009] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, pages 207–220. ACM, 2009. doi:10.1145/1629575.1629596.
- [Leroy, 2009] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi:10.1145/1538788.1538814.
- [Leucker and Schallhart, 2009] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. doi:10.1016/j.jlap.2008.08.004.
- [National Institute of Standards and Technology, 2023] National Institute of Standards and Technology. Artificial intelligence risk management framework (ai rmf 1.0). Technical report, National Institute of Standards and Technology, 2023. doi:10.6028/NIST.AI.100-1.
- [Necula, 1997] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. ACM, 1997. doi:10.1145/263699.263712.
- [Parno *et al.*, 2013] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252, 2013. doi:10.1109/SP.2013.47.
- [RISC Zero, 2026] RISC Zero. RISC Zero zkVM Developer Documentation, 2026. Accessed 2026-05-25. <https://dev.risczero.com/>.
- [Salgado and Wise, 2022] Abner J. Salgado and Steven M. Wise. *Polynomial Interpolation*, pages 231–265. Cambridge University Press, 2022. doi:10.1017/9781108942607.
- [Setty *et al.*, 2023] Srinath Setty, Justin Thaler, and Riad S. Wahby. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, Paper 2023/552, 2023. <https://eprint.iacr.org/2023/552>.
- [Setty *et al.*, 2024] Srinath Setty, Justin Thaler, and Riad S. Wahby. Unlocking the lookup singularity with lasso. In *Advances in Cryptology – EUROCRYPT 2024*, volume 14656 of *Lecture Notes in Computer Science*, pages 180–209. Springer, 2024. doi:10.1007/978-3-031-58751-1_7.
- [Thaler, 2022] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*, volume 4 of *Foundations and Trends in Privacy and Security*. Now Publishers, 2022. doi:10.1561/3300000030.